

PoolPy user guide.

PoolPy is provided as a web interface (<https://trouillon-lab.github.io/PoolPy>) where users can design and compare group testing strategies under defined constraints. The web interface comprises five sections with distinct functions, as follows:

- **Methods Comparison:** The main section of PoolPy, where users can obtain direct comparisons of methods performance and download group testing designs for their applications based on a number of samples and a maximum number of positives.
- **Decoder:** This section provides a tool for decoding results from a group testing experiment. It takes as input the design used and the list of positive groups.
- **Methods & Metrics:** This section describes the methods and metrics used in the other sections.
- **Prevalence:** This section provides a tool for selecting design parameters, such as an expected maximum number of positives and a number of batches, based on the estimated prevalence and an error rate threshold.
- **Automation:** This section provides a tool for generating method files for liquid-handling robots to follow the pooling scheme of a group testing design.

It is also possible to use PoolPy locally without using the web interface, following the instructions in the PoolPy GitHub repository (<https://github.com/trouillon-lab/PoolPy>). Local use might be required in cases that are computationally intensive, such as to generate a design with the random method or to get a full performance summary table for cases that were not precomputed. However, the vast majority of use cases are covered by the web interface, which has precomputed designs across >10,000 conditions, and can generate design on the fly for any numbers of samples and positives for nine out of ten methods. In this **Supplementary Note**, we provide a step-by-step guide on how to use the different sections of the PoolPy web interface to perform a group testing experiment.

1. Determine pooling parameters based on prevalence (optional)

Since many group testing methods adapt their design to an expected number of positive samples, it is important to estimate the prevalence, or hit rate, among the tested samples before choosing a method. This estimated prevalence can then be used to define the pooling parameters. This can be done in the Prevalence section of the PoolPy web interface ([Screenshot 1](#)). There, users can input their test conditions – the number of samples, the estimated prevalence, and the maximum acceptable error – to get the probability of making a potential mistake when reading the results of a pooled experiment; i.e. when more positive samples are present than the defined maximum number of positives. The consequences of such a mistake differ between methods and pooling parameters, ranging from needing to perform a validation round of a few more tests to getting completely inconclusive results, and thus should ideally be avoided.

[Pooling Methods Comparison](#)
[Decoder](#)
[Methods & Metrics](#)
[Prevalence](#)
[Automation](#)

Prevalence

This section is meant to guide the decision of pooling parameters for specific use cases. Particularly, this is to choose a max. number of positive samples and a number of test batches. The two tables reflect cases where all samples are processed into one single batch (left) or broken down into multiple batches (right). The tables show the probability of making at least one mistake while reading the results of either one (left) or multiple combined (right) combinatorial pooling batch(es). Based on a provided prevalence estimate, error rates are reported over ranges of sample (and batch) numbers (S; rows) and of max. number of positive samples (D; columns).

Number of Samples:

Prevalence (as fraction):

Maximum acceptable error:

Enter

Last submitted values:

Number of Samples: 100, Prevalence: 0.005, Max. error: 0.01

Single batch

S \ D	1	2	3	4
100	0.0898	0.0141	0.00167	0.000159
50	0.0261	0.00206	0.00012	5.49e-06
20	0.00447	0.000134	2.84e-06	4.55e-08
10	0.0011	1.46e-05	1.28e-07	7.71e-10
5	0.000248	1.24e-06	3.11e-09	3.13e-12

Corrected for FWER

S \ D	1	2	3	4
1 X 100	0.0898	0.0141	0.00167	0.000159
2 X 50	0.0516	0.00411	0.00024	1.1e-05
5 X 20	0.0222	0.000668	1.42e-05	2.28e-07
10 X 10	0.0109	0.000146	1.28e-06	7.71e-09
20 X 5	0.00494	2.48e-05	6.22e-08	6.25e-11

Red: pooling error > max. error for both single pooling and FWER (all poolings combined).

Yellow: pooling error ≤ max. error for single pooling, but > max. error for FWER (all poolings combined).

Green: pooling error ≤ max. error for both single pooling and FWER (all poolings combined).

Screenshot 1: Pooling parameter determination using the PoolPy Prevalence section.

In the example shown in [Screenshot 1](#), the probability of error is shown for a case with 100 samples with an estimated prevalence of 0.5% across different numbers of batches and maximum numbers of positives (*differentiate*). To keep the probability of making at least one mistake below 1%, several strategies are possible in this example, such as performing a pooling experiment with all samples and a *differentiate* value of 3, or splitting the samples into two batches of 50 samples each and a *differentiate* value of 2. With this overview, users can decide on viable combinations of batch number and *differentiate* values to use to design their pooling experiment in the PoolPy Methods Comparison section.

2. Compare and choose group testing design

With a defined number of samples and maximum number of positive samples, users can compare the methods supported by PoolPy across different key performance indicators using the Methods Comparison section. In this section, precomputed performance summary tables are available for a large range of numbers of samples. In case no precomputed performance summary table is available, specific designs can still be generated and downloaded for nine out of ten methods for any number of samples and positives.

Following the suggestions from the Prevalence section above, we show an example of a method comparison for 50 samples with a *differentiate* value of 2 given by PoolPy

[Pooling Methods Comparison](#)
[Decoder](#)
[Methods & Metrics](#)
[Prevalence](#)
[Automation](#)



Pooling Methods Comparison

This app evaluates the performances of different pooling methods. Designs for specific use cases can be downloaded below.

For details, see the [Methods and Metrics](#) section. For information on the code, visit our [GitHub](#).

To determine a max. number of positives based on prevalence, consult the [Prevalence](#) section.

To decode results from a pooled experiment, consult the [Decoder](#) section.

To generate a method file for a pipetting robot following a pooling design, consult the [Automation](#) section.

Number of Samples:

Max. number of positives:

Enter

Last submitted values:

Max. number of samples: 50, Max. positives: 2

Database reply:

There is a precomputed summary table for 50 samples with up to 2 positives.

To download designs calculated on the fly for your specific values (or locally run the code), follow the section below.

Summary pooling strategy table

Method	Mean experiments	Max samples per pool	N pools	Percentage check	Mean extra experiments	Mean steps
Random	13.43	18.0	11	68	2.43	1.68
multidim-4	13.96	25.0	11	81	2.96	1.81
multidim-3	14.18	16.0	12	75	2.18	1.75
multidim-2	16.38	8.0	15	69	1.38	1.69
Matrix	16.38	8.0	15	69	1.38	1.69
Hierarchical	16.85	10.0	[5 3]	100	11.85	3.0
Ch. Rm. Special	18.0	27.0	18	0	0.0	1.0
STD	25.0	10.0	25	0	0.0	1.0
Ch. Rm. Bktrk	29.0	10.0	29	0	0.0	1.0
Binary	39.6	25.0	6	100	33.6	2.0
Chinese remainder	41.0	25.0	41	0	0.0	1.0

If you need results **quickly**, the best method for you might be the **Ch. Rm. Special**.
 If you mainly need to **reduce test number**, the best method for you might be the **Random**.
 If you **cannot pool many samples** together, the best method for you might be the **multidim-2**.

Screenshot 2: Comparing and selecting group testing design using PoolPy.

(Screenshot 2). In these conditions, different designs are suggested based on different prioritized constraints. To minimize turnaround time, test number or sample dilution, the Chinese Remainder Special, the Random and the matrix designs are suggested, respectively. A summary table is given that details all values for these key metric. Based on these values and suggestions, users can decide on a design of choice and can directly download the corresponding design file from that section. The design file is a table indicating in which pool(s) each sample has to be added.

3. Generate automation file for pipetting robot (optional)

Although the design files can be used directly from the previous section for manual pooling of samples, PoolPy also has an Automation section allowing users to generate method files for pipetting robots to automatize the pooling of samples. In this section,

users can input a design file obtained from the Methods Comparison section (or from the PoolPy python code) and directly download method files for pipetting robots from several of the main manufacturers. The downloaded method file can then be used directly on the corresponding robot to perform the pooling of the samples following the selected design.

4. Decode pooling experiment results

After performing the sample pooling and corresponding experiment, users can decode the results of their tests on the Decoder section of PoolPy. There, users can input their design file obtained from the Methods Comparison section (or from the PoolPy python code) and write which pools were positive (or any other binary characteristic of interest) in their experiment. The decoder then outputs the identity of the positive sample(s) based on the pooled results. In a successful pooling experiment, the decoder will identify the exact set of positive samples. In some cases, e.g. if the number of positive samples exceeds the defined maximum number of positives set by the user, the decoder might not be able to identify the exact set of positive samples, but will rather output a list of possible samples that contains the positive ones.